ANNOFORGE: Bootstrapping Complex Annotation from Guidelines by Recursive Self-Improvement of the Annotation Harness

Anonymous ACL submission

Abstract

Complex annotation scheme requires structural understanding and adherence to detailed guidelines, yet new projects often lack enough data to train a parser. We present ANNOFORGE, which bootstraps an annotator from a guideline and a few examples without fine-tuning. A builder LLM turns the guideline into annotation steps and uses recursive self-improvement to refine instructions, tools, and the workflow from fine-grained error feedback. With just a few dozen annotated sentences, ANNOFORGE improves Chinese and English UMR AnCast++ F by +6.1 and +11.3, respectively, and exceeds the best fine-tuned shared-task system on the official score for the same test documents. On TimeML, refining an explicit linking step reaches 42.4 temporal-awareness F1, above the best TempEval-3 participant under matched evaluation. We further prove that the learned harness transfers unchanged across annotating models.

1 Introduction

Annotating a complex scheme requires not only an understanding of its structures, but also codebooks and conventions given by a guideline. The latter part is the hardest at the start of an annotation project, when examples are scarce. Without enough annotated data, the route of finetuning is closed. Instead, large language models (LLMs) offer a way to bootstrap, whose pretraining gives them a working understanding of linguistic structures and familiarity with structured output formats. Supplying the guideline can direct these capabilities toward the intended annotation scheme.

Prior work has paved this route. Kim et al. (2026) show that reasoning models can apply annotation guidelines and that an iterative moderation process can improve the guidelines from annotation errors, on named-entity recognition, a flat labeling task. Bootstrapping a complex ontology makes two further demands. a) Guidelines can be lengthy: the TimeML manual spans 71 pages (Saurí et al., 2006), and the guidelines of Uniform Meaning Representation (UMR) (Van Gysel et al., 2021; Bonn et al., 2024) run to 366 thousand characters, more than three times as long, and b) annotation may also require external resources, such as PropBank frames (Palmer et al., 2005) for UMR. Models use long contexts unevenly, a phenomenon known as "lost in the middle" (Liu et al., 2024), and agents lose details of long procedural instructions (Panavas et al., 2026).

We propose ANNOFORGE, decomposing the guideline into a chain of steps and equipping those steps with tools, then improving this *harness* through recursive self-improvement (RSI) (Figure 1). A builder LLM derives a seed harness from the guideline; fine-grained feedback on a few annotated examples then guides revisions to step instructions, the workflow, and tools. The harness organizes the model's linguistic judgments and supports them with resource lookups and executable rules.

Our contributions are threefold:

1. We introduce a guideline-and-harness optimization framework for bootstrapping complex annotation, with editable instructions, steps, and tools (§3).
2. ANNOFORGE can achieve state-of-art results from a guideline and a few dozen annotated sentences, and harness refinement yields even larger gains across models (§5.1, §5.2).
3. The learned harness generalizes and transfers to other models and unseen text. (§5.3, §5.4).

2 Related Work

LLMs as annotators. Prior work uses LLMs as annotators for text labeling (Gilardi et al., 2023; He et al., 2024). Closely related studies improve

annotations by refining guidelines from errors or annotator disagreements (Kim et al., 2026; Kim and Yoon, 2026). These studies use flat labeling tasks, mainly named-entity recognition. We extend the approach to complex annotation by refining the whole harness.

Recursive self-improvement. LLMs can reflect on feedback to revise their outputs and behavior (Madaan et al., 2023; Shinn et al., 2023). Recursive self-improvement extends such loops to agent code (Zelikman et al., 2024), including the evolution of harness instructions, tools, and workflows (Lee et al., 2026; Zhang et al., 2026; Lin et al., 2026). We use fine-grained annotation errors to guide this evolution.

3 ANNOFORGE

3.1 Overview

Annotator. An annotator is a harness $H = (S, T, G, R)$ run by language models: *skills* S, the instructions of each model step; *tools* T, typed Python functions a step may call; a *chain* G, the order of the steps and the model that runs each; and *rule steps* R, code that rewrites an annotation without a model. Each step receives the input text, the current annotation, and unresolved findings from earlier steps. A *rewrite* step produces a complete annotation, a *check* step identifies problems and proposes fixes for the next rewrite step, and a rule step runs code. The final annotation is the last one that parses. Every step receives the same output format; all other instructions are learned.

Seed. The builder model reads the guideline and writes the seed harness H_0: three steps, `generate` (writes the annotation), `critique` (lists problems) and `finalize` (rewrites the annotation with the findings), each a checklist of at most 8,000 characters. The builder sees no annotated data, and the seed has no tools.

Recursive self-improvement. Training data come as groups of five consecutive sentences of one document. Round n turns H_{n-1} into H_n with group n (§3.2): annotate the group, score it with the judge, let the builder summarize the problems, reflect on their causes and propose operations on skills, steps and tools, admit the operations that pass machine checks, and annotate the group again. Each H_n is tested on held-out documents, which the builder never sees.

Judge. The builder needs to know which part of the scheme each error belongs to. A judge aligns the predicted annotation with the gold one and scores a set of checks, each covering one part of the scheme (for example, node concepts, argument roles, attribute values, or one type of link). It returns *units*, each with a verdict (correct, wrong, missing, extra), the sentence, and the gold and predicted elements. The judges we use are described in §4.2.

3.2 The harness and its improvement

Structure. A harness specifies an ordered chain of model and rule steps. Each model step has a skill, a model assignment, and a set of callable tools; each rule step has a Python function. The generator G runs `generate` and `finalize`, and the critic C runs `critique`; a new model step is assigned to either role. The chain is stored in YAML, skills in Markdown, and tools and rules in Python. Round n copies H_{n-1} and edits the copy to produce H_n, preserving earlier versions.

Resources. The annotating steps see only their input. Tools read resource files through one module. The builder receives a manifest of these files (for example lexicons, lists of labels and the guideline itself) and may open them while it reflects. It receives no instruction about what to build: the seed has no tools, and every tool is the builder's own design.

One round. (1) *Annotate*: H_{n-1} annotates the five sentences of group n; the sentences before the group are given with their gold annotations as context. The trace keeps every step's output and findings. (2) *Judge*: the judge scores the group and lists the units that are not correct. (3) *Reflect*: one builder call receives the output format, the harness (chain, all skills with their sizes, all tools), the resource manifest, the changes and training scores of earlier rounds (never a test score), and, for each sentence, its gold annotation, every step's output and its incorrect units. It returns *problems* (error patterns with evidence and the step to fix), a *reflection* (why they happen and what to change), *operations*, and *suggestions* that the operations cannot express. During the call the builder can list and read resource files and search the guideline. (4) *Admit*: operations are applied in order to a copy of H_{n-1} when they pass the machine checks. (5) *Re-annotate* the group with H_n, and test H_n on held-out documents.

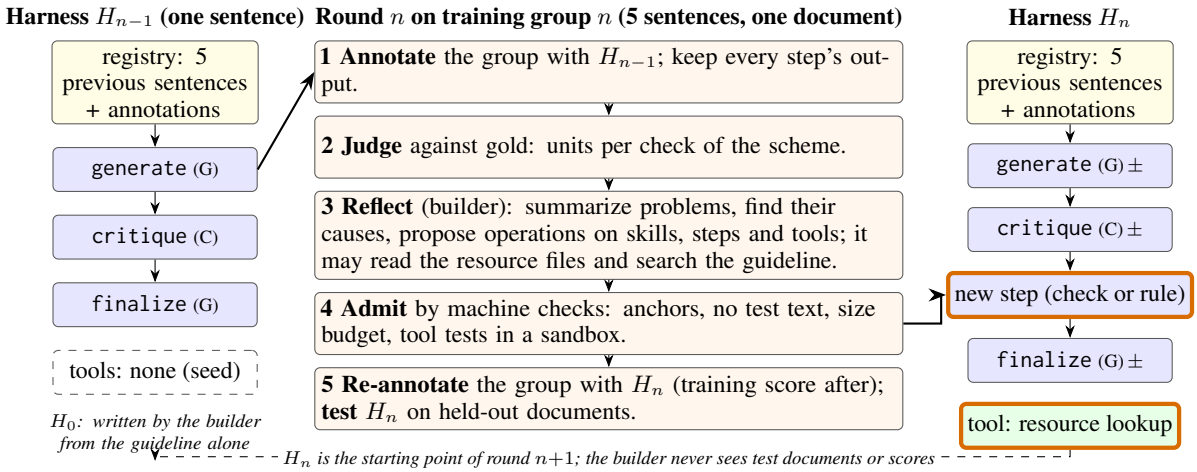


Figure 1: AnnoForge. Left: the seed harness annotates one sentence at a time; each step sees the previous five sentences with their annotations. Middle: one round of recursive self-improvement on a group of five gold sentences. Right: the next version; skills are edited (±), and steps and tools may be added (orange), e.g. a lookup over resource files that the builder writes on its own (§5.4). G and C are the generator and critic models of a configuration.

Reflection. The builder turns the judge's error patterns into changes to the harness. It can edit or rewrite skills, add or remove model and rule steps, create, revise, or remove tools, and change which tools a step may call. A local text edit must identify an anchor that occurs exactly once; a full rewrite consolidates accumulated instructions.

Proposed changes are applied when they pass machine checks. Skills may contain at most 10,000 characters, and the chain at most five model steps; a new step follows an existing one, and the first step remains a rewrite step. Added text may not copy spans of test sentences (eight characters in scripts without word spaces, six words otherwise). Tools and rules must use whitelisted imports, avoid direct file access, code evaluation, and introspection, define a documented function with type annotations, and pass at least three tests in a separate process. Failed code is returned to the builder for one repair attempt. These checks determine which proposed changes become part of the next harness, without a reviewer model.

4 Experiments

4.1 Benchmarks

UMR. Uniform Meaning Representation (Van Gysel et al., 2021; Bonn et al., 2024) combines sentence-level concept and argument graphs, word alignments, and document-level temporal, modal, and coreference relations. Its guideline has 365,742 characters. Resources include Chinese and English PropBank frames (Xue and Palmer, 2009; Palmer et al., 2005), lists of rolesets, relations and attribute values, and the guideline itself. We evaluate Chinese and English annotation using the first UMR parsing shared task (Štěpánek et al., 2026), released with UMR 2.2 (Bonn et al., 2026).

TimeML. TimeML (Pustejovsky et al., 2003) requires recognizing events and time expressions, assigning their attributes and normalized values, and linking them with fourteen temporal relation types within and across sentences. We follow the 71-page TimeML 1.2.1 guideline (Saurí et al., 2006), which has 115,146 characters and is the only resource file, and evaluate on TempEval-3 (UzZaman et al., 2013). Outputs use inline entity tags followed by temporal links. Data selection and training/test splits for both benchmarks are detailed in Appendix A.

4.2 Judges and evaluation

UMR. We use umrjudge for feedback: AnCast++ (Sun and Xue, 2024; Sun et al., 2025) aligns predicted and gold nodes, and 25 checks in 11 groups compare concepts, senses, attributes, arguments, and document-level relations (Appendix B). We report AnCast++ sentence-graph F (F, the main metric) and its comprehensive score including document-level relations (*Comp.*), after the cleanup of Chun and Xue (2024), macro-averaged over documents. We also report the shared task's official juːmæʧ score (Štěpánek et al., 2026; Zeman and Gamba, 2026), computed with `compare_umr.pl`

against the official test gold. It maps nodes through token alignment and therefore also scores alignment; its gold differs from the cleaned gold used by AnCast++ (Appendix A). Confidence intervals use a paired document bootstrap with 20,000 resamples; we also report a sentence-level bootstrap of AnCast scores. Fresh-sample repetitions of H_0 and H_{10} measure run-to-run variation.

TimeML. We built timemljudge using TempEval-3's extent and attribute measures for events and times (UzZaman et al., 2013) and by extending TempEval-2's categories of temporal links (Verhagen et al., 2010) to event–time and event–event within or across sentences, event–document time, and time–time. Each category has separate checks for which pairs are linked and whether their relations are correct. For overall evaluation we use the official TempEval-3 scorer, reporting temporal-awareness TLINK F1 (*TLINK*, the main metric), event and time-expression extent F1, and time-expression value F1. Confidence intervals use 20,000 paired document bootstrap resamples, recomputing corpus-level TLINK F1 from per-document counts. Both judges' per-check scores support the analysis.

4.3 Setup

Models. The annotators are Claude Haiku 5.5 and GPT-6.1-sol. Claude Sonnet 5.5 writes the seed and is the reflector and editor in every round. Claude models run with adaptive thinking at the Messages API's default effort (Haiku medium, Sonnet high), GPT with medium reasoning effort. Each configuration was run once, with cached answers, apart from the UMR repetitions described above.

Baselines and learned harnesses. We compare three annotation approaches. The baseline *guideline prompt* makes one call per sentence with the whole guideline and the sentence, using Haiku, Sonnet or GPT, without earlier sentences or examples. The *seed harness* H_0 uses the three steps written from the guideline, with all four Haiku/GPT generator–critic pairs and with Sonnet in both roles. ANNOFORGE runs ten rounds of RSI with Haiku in both roles, evaluating $H_0, \ldots, H_{10}$. We then transfer H_{10} unchanged to the other generator–critic pairs and to Sonnet; pairwise transfer comparisons use the same two documents per benchmark. For TimeML, the main trajectory trains on platinum documents outside the test set and a second trajectory on TBAQ-cleaned. All approaches receive the same output-format instructions.

Guideline access and harness ablations. To compare ways of applying the guideline, the *guideline-through-tools* variant supplies only its address and lets the model search and read it, together with UMR resource files; Haiku runs on all test documents and GPT on two per benchmark. To isolate instruction refinement, a *guideline-only round* starts from the guideline prompt and permits only edits or rewrites, with no steps, tools, rules, or length limit. To isolate harness components, we run two *restricted RSI* variants for two rounds each: one disables tool operations, the other rule steps. For TimeML we also test an explicit linking step (§5.4).

Prior systems. We re-score earlier systems on exactly our test documents from their released outputs: the two UMR shared-task systems (Heinecke and Asadullah, 2026; de Vergnette and Amblard, 2026), fine-tuned on the shared task's training data, of which Orange is best on both Chinese and English; and the TempEval-3 end-to-end systems (UzZaman et al., 2013), of which ClearTK-2 (Bethard, 2013) is best on temporal awareness and is the best earlier system we compare with. The organizers also report TIPSem (Llorens et al., 2010) as a reference that scores higher, and they separate it from the participants because some annotators used it to pre-label the platinum data (UzZaman et al., 2013, §2.2, §5); on our documents 73% of the gold TLINK pairs are pairs that TIPSem links (ClearTK-2: 26%). We report TIPSem as a reference, not as the state of the art.

5 Results

5.1 Main results

Table 1 gives every system on the same test documents.

A long guideline in the prompt is not enough. Given the whole guideline and the sentence, Haiku reaches 30.9 AnCast++ F on Chinese UMR and Sonnet 50.1; on English the prompt baselines are stronger (50.8 and 54.1). Their errors follow the guideline's own notation where the data use another: modal strength written as `:modstr FullAff` inside the graph instead of document-level relations such as `(author :full-affirmative e)`, capitalized attribute values, `:ref-number` for `:refer-number`. Every prompt baseline has a document-level modal score of 0.

	UMR Chinese			UMR English			TimeML		
System	F	Comp.	ju:mætʃ	F	Comp.	ju:mætʃ	TLINK	Event	Value
Fine-tuned and feature-based systems (re-scored on our test documents)									
UMR shared task: Orange	37.7	34.0	38.8	40.3	24.8	27.3	–	–	–
UMR shared task: Sema	34.7	33.4	29.2	32.1	21.5	21.4	–	–	–
TempEval-3: TIPSem†	–	–	–	–	–	–	(46.3)	(82.4)	(72.5)
TempEval-3: ClearTK-2	–	–	–	–	–	–	29.3	76.9	68.0
TempEval-3: NavyTime	–	–	–	–	–	–	27.7	82.1	70.9
Guideline in the prompt, one call per sentence									
Haiku 5.5	30.9	24.1	12.5	50.8	29.9	21.7	30.5	81.5	82.6
Sonnet 5.5	50.1	38.0	24.4	54.1	31.8	28.4	35.1	83.2	80.4
GPT-6.1-sol	51.9‡	38.0‡	28.5‡	73.4‡	60.4‡	43.8‡	39.5	82.8	78.2
Seed harness H_0 (generator / critic)									
Haiku / Haiku	46.4	34.0	24.1	40.9	22.5	25.9	39.5	83.8	78.6
Haiku / GPT	46.3	33.8	25.8	43.0	24.2	26.6	38.1	81.4	70.3
GPT / Haiku	47.8	34.7	23.7	42.3	23.5	26.2	39.6	82.0	75.0
GPT / GPT	49.1	35.5	24.4	42.1	23.8	26.0	39.4	80.8	72.7
Sonnet / Sonnet	48.1	35.1	24.3	43.5	23.6	27.1	38.9	80.0	78.5
Learned harness H_{10} (ANNOFORGE, built with Haiku / Haiku)									
Haiku / Haiku	52.4	47.2	42.9	52.2	33.0	38.0	39.7	84.0	79.3
Haiku / GPT (transfer)	52.8‡	46.5‡	41.8‡	53.7‡	48.0‡	41.8‡	29.0‡	80.3‡	73.7‡
GPT / Haiku (transfer)	53.2‡	47.4‡	40.4‡	55.3‡	48.7‡	40.7‡	38.1‡	81.3‡	82.3‡
GPT / GPT (transfer)	56.4	49.7	42.4	57.1‡	50.0‡	40.8‡	n/a	n/a	n/a
Sonnet / Sonnet (transfer)	58.9	51.2	42.5	53.1	33.4	38.7	39.5	84.0	78.5
Haiku / Haiku, TimeBank-trained	–	–	–	–	–	–	23.8	84.2	84.1

Table 1: All systems on the test documents (UMR: 99 Chinese and 95 English sentences of the shared task's test set; TimeML: 96 TempEval-3 platinum sentences). F: AnCast++ sentence-graph F; Comp.: AnCast++ comprehensive score with document-level relations; ju:mætʃ: the shared task's official score against its test gold. TLINK: TempEval-3 temporal awareness F1; Event: event extent F1; Value: TIMEX3 value F1. Transfer: H_{10}, built with Haiku as generator and critic, run unchanged with other generator/critic models (§5.3). ‡: GPT-6.1-sol runs stopped by the API budget, scored on the test documents they finished: the guideline prompt on 4 Chinese documents (41 sentences) and 1 English document (9 sentences), the transfer on two documents per benchmark (22 Chinese, 38 English and 22 TimeML sentences; Appendix Table 8). These cells cover fewer documents than the others. n/a: no test document finished. †TIPSem (Llorens et al., 2010) is the organizers' reference, not a participant: some annotators used it to pre-label the platinum data, and the organizers separate it from the participants for that reason (UzZaman et al., 2013); 73% of the gold TLINK pairs of our documents are pairs TIPSem links, against 26% for ClearTK-2, the best participant. Its TLINK score is a reference, not the state of the art.

The seed harness helps where the guideline is hard to apply. Written from the same guideline, the three-step seed improves over the Haiku prompt baseline on Chinese UMR (46.4 vs. 30.9) and on TimeML (TLINK 39.5 vs. 30.5), where it is ten points above the best earlier system on our documents (ClearTK-2, 29.3); only the organizers' reference TIPSem scores higher (46.3), whose output the platinum gold was partly corrected from (§4). On English UMR, where the prompt baseline is strong, the seed, which compresses the guideline into 18K characters of checklists, stays below it (40.9 vs. 50.8).

RSI improves the harness on held-out documents. After ten rounds of five sentences each, H_{10} improves over the seed on both UMR test sets: Chinese F 46.4 → 52.4 (paired document bootstrap +6.1, 95% interval +1.0 to +10.1; sentence level +7.3, 65 of 98 sentences better and 25 worse by more than a point) and English 40.9 → 52.2 (+11.3, +7.7 to +14.9; 72 of 95 better, 18 worse). The comprehensive score, which adds the document level, rises more (34.0 → 47.2; 22.5 → 33.0), and so does the official ju:mætʃ score against the shared task's own gold (24.1 → 42.9; 25.9 → 38.0). H_{10} is above the best shared-task system on the same documents on every UMR metric (ju:mætʃ 38.8 and 27.3; F 37.7 and 40.3); that system was fine-tuned on the shared task's training data, and H_{10} learned from 50 annotated sentences. Two independent repetitions of H_0 and H_{10} with fresh samples differ by 0.9/2.1 (Chinese) and 1.2/1.5 (English) F, well below the gains. On TimeML, temporal awareness stays level (39.5 → 39.7) while the linked pairs

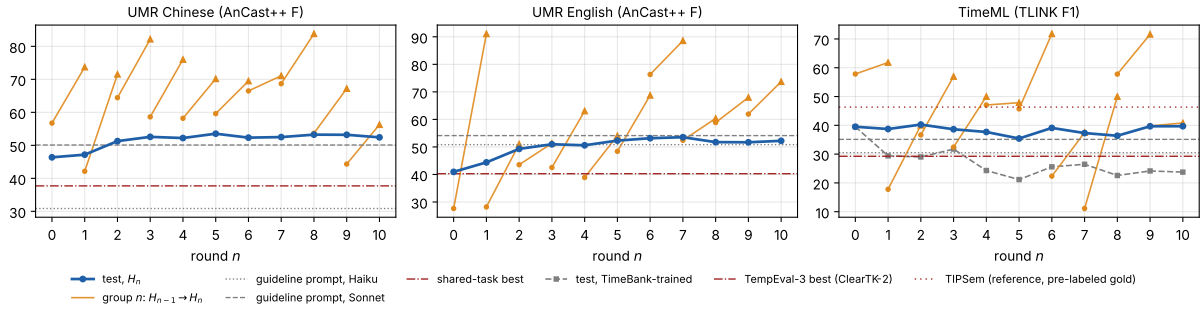


Figure 2: Ten rounds of ANNOFORGE with Haiku as generator and critic. Blue: test score of H_n. Orange: training group n, annotated by H_{n-1} (circle) and, after the round, by H_n (triangle). Grey squares (TimeML): the trajectory trained on TimeBank/AQUAINT instead of platinum documents. Reference lines: guideline-in-prompt baselines (Haiku, Sonnet) and the best earlier system on the same test documents (TimeML: ClearTK-2, the best TempEval-3 participant; the organizers' reference TIPSem, which some annotators used to pre-label the platinum data (UzZaman et al., 2013), is drawn separately).

change (§5.4).

5.2 Guideline or harness?

Is the harness needed, or would a better guideline do? Table 2 compares, with Haiku in every role, ways of giving the model the guideline.

Reading the guideline through tools is worse than having it in the prompt. With only the guideline's address and tools to search and read it, Haiku falls below the prompt baseline everywhere, most on Chinese (F 14.9 vs. 30.9). It looks things up (69 %, 82 % and 24 % of its calls use a tool), but only what it knows it needs; the conventions it is missing go unread, and on Chinese it writes English concepts (`define-01`, `spaceflight`) for Chinese words. A stronger reasoning model reads better: on the two documents per benchmark of Appendix Table 8, GPT-6.1-sol with tools reaches 53.3 (Chinese) and 62.1 (English) F, as high as H_{10} with Haiku (51.4, 52.3) or with GPT (52.4, 57.1) on the same documents; the harness keeps its lead on the Chinese document level and on the official score (comprehensive 39.3 vs. 45.1, juːmæʧ 31.0 vs. 39.2).

Editing the guideline fixes conventions, not structure. One round of reflection that may only edit the guideline prompt appends a block of "house conventions" to it (4–7K characters, at the end or right after the first line of the 366K-character guideline). This is what the harness's first round also learns, and it pays where conventions are the problem: juːmæʧ, which scores alignment and notation, rises from 12.5 to 36.3 on Chinese and from 21.7 to 31.8 on English, and the comprehensive score rises on both. On English, where the prompt baseline is already strong, F reaches 52.5, as high as H_{10} (52.2). On Chinese, F barely moves (30.9 → 32.7) while the harness after the same round is at 47.2, and on TimeML the edited guideline (TLINK 35.5) stays below the seed harness (39.5). The difference is the decomposition: a guideline edit states the data's notation, and the harness also splits the work into a draft, a check against the guideline and a revision, and gives the model lookups. Where the model's single pass is already good (English), conventions are most of what is missing; where it is not (Chinese), the steps are.

5.3 What RSI learns carries over

What a round learns can carry over in two ways: to documents the builder never saw, and to annotating models other than the one it watched. We call the second *transfer*.

Each round improves its own training group, by +14.9 (Chinese), +19.1 (English) and +16.0 (TimeML) points on average (Figure 2, orange). For UMR these gains reach the test documents: the test score rises in most rounds (6 of 10 for Chinese, 7 for English), and across rounds the training gain and the test gain are positively correlated (Spearman ρ 0.58 and 0.50; with ten rounds, p 0.08 and 0.14). The test curves rise for three to six groups, 15–30 annotated sentences, and then flatten (best 53.5 at round 5 for Chinese, 53.5 at round 7 for English), while the training groups keep improving: later rounds increasingly fix what is particular to a group, a narrative, a list of bond prices, a phrasal verb. Edits are kept without a held-out check, and

Haiku in every role	UMR Chinese			UMR English			TimeML		
	F	Comp.	juːmæʧ	F	Comp.	juːmæʧ	TLINK	Event	Value
Guideline in the prompt	30.9	24.1	12.5	50.8	29.9	21.7	30.5	81.5	82.6
Guideline through tools (link only)	14.9	12.1	8.8	46.2	27.5	19.2	27.4	70.9	75.9
Guideline + 1 round (edits only)	32.7	30.0	36.3	52.5	34.3	31.8	35.5	80.5	85.4
Seed harness H_0	46.4	34.0	24.1	40.9	22.5	25.9	39.5	83.8	78.6
Harness H_1	47.2	42.4	42.0	44.4	28.4	34.3	38.7	82.4	84.7
Harness H_{10}	52.4	47.2	42.9	52.2	33.0	38.0	39.7	84.0	79.3

Table 2: The guideline against the harness. *Through tools*: the instructions give only the guideline's address; the model can search and read the guideline (and, for UMR, the resource files) with tools. *+ 1 round*: one round of RSI on the first training group in which the builder may only edit the guideline prompt (no steps, tools or rules, no length limit); H_1: the same round on the harness.

G / C	UMR zh (F)			UMR en (F)			TimeML (TLINK)		
	H_0	H_{10}	Δ	H_0	H_{10}	Δ	H_0	H_{10}	Δ
Haiku / Haiku	46.4	52.4	+6.1	40.9	52.2	+11.3	39.5	39.7	+0.2
Haiku / GPT	38.4‡	52.8‡	+14.5‡	43.1‡	53.7‡	+10.6‡	31.3‡	29.0‡	−2.3‡
GPT / Haiku	42.4‡	53.2‡	+10.8‡	42.4‡	55.3‡	+12.8‡	34.3‡	38.1‡	+3.8‡
GPT / GPT	49.1	56.4	+7.3	42.7‡	57.1‡	+14.4‡	39.4	n/a	n/a
Sonnet / Sonnet	48.1	58.9	+10.7	43.5	53.1	+9.6	38.9	39.5	+0.6

Table 3: Generator (G) and critic (C) models (H: Haiku, G: GPT-6.1-sol, S: Sonnet). H_{10} was built with Haiku in both roles; for the other pairs it is used as it is. ‡: on the two test documents per benchmark that the GPT runs finished (22 Chinese, 38 English and 22 TimeML sentences), H_0 and H_{10} on the same documents; n/a: no test document finished.

Harness (Haiku / Haiku)	UMR Chinese			UMR English		
	F	Comp.	juːmæʧ	F	Comp.	juːmæʧ
H_0 (seed)	46.4	34.0	24.1	40.9	22.5	25.9
H_{10}	52.4	47.2	42.9	52.2	33.0	38.0
H_{10} without tools	53.0	48.4	43.5	50.6	32.2	36.6
H_{10} without rule steps	54.3	49.0	43.1	52.6	32.6	37.5
H_{10} with the seed's skills	47.4	43.4	28.7	40.0	21.9	24.9

Table 4: What the parts of H_{10} contribute: the learned harness without its tools, without its rule steps, and with the seed's instructions in place of the learned ones (tools and rules kept).

three rounds per language lower the test score, by at most 1.8 points.

Across models (transfer). H_{10} was built with Haiku as generator and critic: the builder saw only Haiku's errors. In a transfer run, H_{10} is used unchanged (the same skills, steps and tools) and only the models that run its steps are replaced; each transfer run is compared with the seed H_0 run by the same models (the rows marked "transfer" in Table 1). If RSI only patched Haiku's weaknesses, the other models would not gain. They gain as much: with GPT-6.1-sol as generator and critic, Chinese F rises from 49.1 (seed) to 56.4 (comprehensive and juːmæʧ as with Haiku, Table 1); on the 38 sentences of the two English documents the GPT run of H_{10} completed within our API budget, from 42.7 to 57.1 (Haiku: 40.5 → 52.3 on the same documents); with Sonnet in both roles, Chinese 48.1 → 58.9 and English 43.5 → 53.1. On two documents per benchmark we also ran the mixed pairs (Appendix Table 8): on UMR every pair gains 7 to 14 F from H_{10}, and the four pairs end within five points of each other; on TimeML, whose temporal-awareness score on 22 sentences moves by several points between runs, the pairs neither gain nor lose consistently. What the builder wrote is about the scheme, not about the model it watched.

Generator and critic. At the seed, the choice of models matters much less than the harness (Table 3): the four Haiku/GPT pairs are within three points of each other on every benchmark, close to the run-to-run variation, while RSI adds 6 to 11. A stronger generator helps a little on Chinese (GPT/GPT 49.1 vs. Haiku/Haiku 46.4); swapping only the critic changes little (Haiku/GPT 46.3, GPT/Haiku 47.8), and the mixed pairs show no complementarity beyond that. Consistently, the builders' main complaint about the critic was not its model but its instructions: it "fixed" correct annotations (§5.5).

5.4 How the harness improves

Conventions first, then lookups. Figure 3 breaks the gains down by judge check (Appendix Tables 13–15). For UMR, the largest gains are on conventions that the guideline states differently from the data or not at all: the document-level format and modal relations (Chinese modal 0 → 50, English 0 → 32), attribute names and values, entity concepts, granularity. The next largest are PropBank senses (Chinese 88 → 94, English 67 → 80) and core arguments of events (52 → 63,

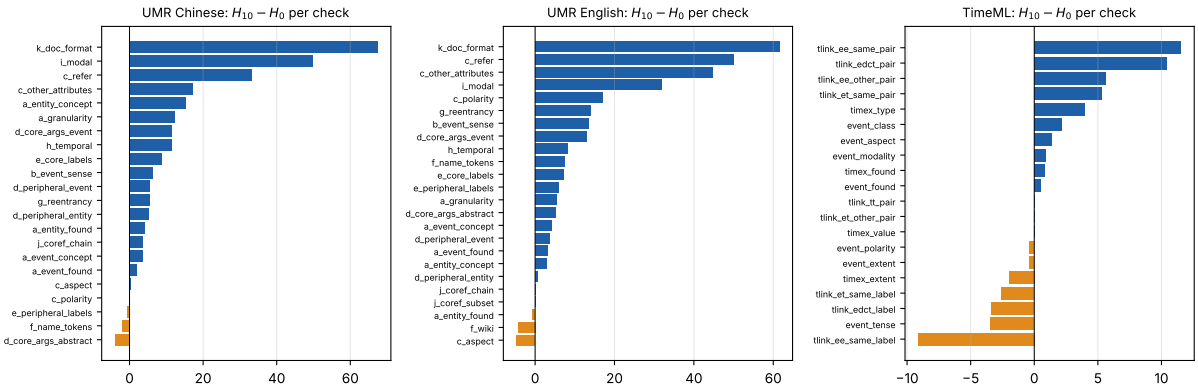


Figure 3: Change of each judge check from H_0 to H_{10} on the test documents (checks with at least six units; Haiku / Haiku; TimeML: platinum-trained trajectory).

42 → 55). Aspect and coreference change little, and document-level temporal relations stay low (Chinese 3 → 15, English 5 → 13). The ablation (Table 4) separates the parts: the learned instructions carry most of the gain (with the seed's instructions in place of the learned ones, F falls by −5.0 and −12.3); removing the tools hardly changes F (+0.6, −1.6) but costs exactly the checks the tools serve, senses (Chinese 94 → 78, English 80 → 73) and core arguments (63 → 55, 55 → 47); removing the rule steps changes little (+1.9, +0.4): the learned instructions alone already produce most of what the rules enforce. With one run per configuration, differences of one or two points are within the run-to-run variation measured above.

Restricting the loop. Two rounds of RSI from the same seed with restricted operations (Appendix Table 9) agree with this ablation. Without rule steps the builder ends close to the full loop (Chinese 51.6 vs. 51.3 F at H_2, English 47.0 vs. 49.3), and on Chinese it writes the PropBank tool one round earlier. Without tools, Chinese falls behind (48.6), most on the official score, and English does not (49.7): in the first rounds the builder mostly fixes conventions, which instructions can carry, and the tool pays for itself on Chinese senses.

TimeML: pairs, not relations. The fine-grained judge shows what the flat temporal-awareness score hides. The platinum-trained harness links the right pairs much more often: event–document time from 34 to 45, event–event within a sentence from 43 to 55, across sentences from 6 to 12 (pair F); but the relation of a pair linked on both sides gets worse (event–event within a sentence 75 → 66), and temporal awareness needs both. Which pairs to link is a convention of an annotation project; which relation holds is a semantic judgment. Time expressions and events, already near the 2013 systems at H_0, change little.

An explicit link step. The judge locates TimeML's problem in which pairs are linked: the seed links events to the document time that the gold leaves unlinked (123 of its 162 event–DCT links), misses most within-sentence event pairs (it links 63 of 174) and gets almost no pair across sentences right. We therefore wrote, by hand, a fourth step that rebuilds a sentence's links as a sparse backbone (a main event, one anchor per other event, the main event to the previous sentence's main event and to the document time) and a rule that links each time expression to the document time from its normalized value (Table 5). Each part on its own makes temporal awareness worse: the rule adds time–DCT links that the gold mostly does not have (only 23 of the 138 time expressions of the platinum documents are linked to the DCT, with no pattern of type or value), and the backbone links many pairs across sentences that the gold does not (36.0, and 34.0 when cross-sentence links are restricted to explicit connectives, against 39.5); added to H_{10}, the same step lowers it too (38.4, 37.3). Given to RSI, the step becomes something the builder can tune. In the first round on the platinum training group it rewrote the step after the gold's practice (dependents of a clause to its main event, the main event both to its time expression and to the document time) and reached 42.4, the best TimeML harness in our experiments (recall 58.3), above the ten-round H_{10} (39.7) and 13 points above the best earlier system (ClearTK-2, 29.3); in the second round it reduced the over-linking across sentences and lost part of the gain (40.4). A dedicated step

	TLINK			E–DCT	E–E sent.	E–E across	T–DCT
Harness (Haiku)	F1	P	R	gold pairs / linked pairs (gold: 65, 174, 73, 15)			
H_0 (seed)	39.5	40.0	39.0	39/162	63/119	4/53	0/0
H_0 + DCT rule	37.3	36.4	38.2	41/162	60/107	3/55	11/46
H_0 + link step + DCT rule	36.0	28.1	50.3	20/59	60/128	32/199	8/39
H_0 + link step (revised)	34.0	30.9	37.7	25/72	62/132	3/109	0/0
+ RSI round 1	42.4	33.3	58.3	37/105	98/165	34/209	0/0
+ RSI round 2	40.4	34.6	48.4	32/97	84/147	27/160	0/0
H_{10}	39.7	42.6	37.2	32/78	84/134	8/61	0/0
H_{10} + link step + DCT rule	38.4	32.5	46.8	25/64	61/122	32/152	8/35
H_{10} + link step (revised)	37.3	37.2	37.4	26/77	65/125	2/71	0/0

Table 5: An explicit TLINK step for TimeML. The link step rebuilds the links of each sentence as a sparse backbone (main event, one anchor per event); the DCT rule links every time expression to the document time from the normalized values. Right: of the pairs the harness links, how many the gold also links.

gives the loop a place to learn the gold's linking practice.

The builder learns the gold it sees. Trained on TimeBank and AQUAINT instead (grey in Figure 2), whose temporal links are much sparser than the platinum gold and where several five-sentence windows have no link at all, the harness learns to link less: TLINK precision rises (40.0 → 45.7), recall falls (39.0 → 16.0), and temporal awareness drops to 23.8, while time values improve (78.6 → 84.1). The loop does what it is built to do, adapt the annotator to the gold it is given; it cannot tell a project's convention from an accident of a corpus. The builders' suggestions, which the loop never applies, point the same way: 65 of the 75 suggestions of the three main trajectories are about the gold, 34 of them explicitly about inconsistencies (e.g. `:refer-number` on plural nouns in some English documents but not in others), which a human moderator should resolve.

The builders invent lookup tools. The UMR builders were told only that PropBank files exist, and each wrote a lookup in the first rounds: `lookup_frame` over the Chinese frame files (round 2), which also answers "no frame" with the UMR convention for such predicates, and `roleset_lookup` over the English argument descriptions (round 1), which the builder replaced in round 9 (and refined again in round 10) after noticing that the annotator chose wrong senses from role descriptions alone (the new version reads the XML frames with sense names and falls back from *researchers* to *research*). With H_{10}, 60 % (Chinese) and 39 % (English) of Haiku's calls use the tool. The TimeML builder, whose only resource was the guideline, wrote no tool. Rule steps were added more often (10 for Chinese, 4 for English): the builders turned recurring convention errors into code that normalizes modal labels and constants, moves modal strength from the graph to the document level, and repairs variables and alignments. No new model step was proposed: when one kind of decision needed its own pass, the builders preferred code, or a new section in an existing skill, to an extra call per sentence. Table 6 counts every operation.

Compact by construction. Most operations modify or add instruction text; when a skill approached the 10,000-character budget, the builder consolidated it (7 rewrites for Chinese, 3 for English). The final harnesses have 26.4K (Chinese), 27.9K (English) and 28.8K (TimeML) characters of instructions, against 366K and 115K for the guidelines (Appendix Tables 10–12).

5.5 Case study: one trajectory

The English trajectory shows the order in which the builder worked (Appendix H). Round 1 found that the annotator wrote the guideline's notation (`:ref-number Singular`, modal labels such as `FullAff`, tense metanodes in temporal relations), rewrote the three skills in the data's notation, and added the PropBank tool. Rounds 2–6 worked on the document level: the `(root :modal author)` relation, modal relations for every event with aspect, coreference direction for plural pronouns, and rule steps that clean the document-level graph. Rounds 7–10 turned to semantics: aspect cues such as habitual *would*, phrasal verbs as one roleset (`pick-up-04`), the sense distinctions that led to the new lookup. The critic was a recurring problem: in several rounds the builder found that `critique` had "fixed" correct annotations and `finalize` had ap-

Benchmark	edit_skill add	edit_skill modify	edit_skill delete	rewrite _skill	add _step	delete _step	add _tool	replace _tool	delete _tool	set step _tools
UMR Chinese	20/22	32/33	14/15	7/7	10/10	0	1/1	0	0	0
UMR English	24/27	32/34	18/18	3/3	4/7	0	1/1	2/2	0	0
TimeML	26/30	68/74	27/29	0	0/1	0	0	0	0	0

Table 6: Operations applied / proposed over the ten rounds, by type.

plied the fixes, and restricted the critic's checklist, in line with the finding that self-correction without an external signal often hurts (Huang et al., 2024; Kamoi et al., 2024).

6 Conclusion

Complex annotation can be bootstrapped from a guideline and a few examples. Without fine-tuning, an LLM that receives the guideline as a chain of steps, tools over the project's resources, and a loop that revises them against a handful of gold sentences annotates UMR better than the fine-tuned systems of its shared task on the same documents. The gains come from the harness: its decomposition into steps, its tools, and recursive self-improvement guided by a fine-grained judge, which plays the role that moderation plays in a human annotation project. Editing the guideline corrects notation; the steps and tools improve the annotation itself. In the loop, the builder model learned to edit the guideline, to write the tools the guideline presupposes, and to turn recurring decisions into code.

The loop learns the conventions of the gold it sees. It learned which TimeML events to link, learned to link less from a sparser corpus, and, given an explicit linking step, learned the gold's linking practice in one round. Senses and arguments improved where a tool supplied the frames; aspect, coreference and temporal relations, which no resource supports, moved little; and the builder's own suggestions point mostly to inconsistencies in the gold. This suggests a division of labor for annotation projects. The guideline becomes an executable object, tested and revised against data from the first annotated sentences on: the model maintains the instructions, tools and checks, and people decide the conventions and adjudicate the inconsistencies it surfaces. A new scheme or language then starts from a working annotator.

Limitations

Scale and variance. Each test set has at most 100 sentences in four to seven documents, and every configuration was run once, in one order of training groups, with cached answers. The bootstrap intervals describe variation over test documents, not over runs of the builder or of the annotators; repeated trajectories and other group orders are needed to estimate that variance.

Models. The builder (Sonnet) and the main annotator (Haiku) are from one model family. Within our API budget for GPT-6.1-sol (through OpenRouter, without prompt caching) we ran all seed configurations with GPT, the GPT guideline-in-prompt baseline on all TimeML documents and on 4 Chinese and 1 English document, the transfer of H_{10} to GPT/GPT on all Chinese documents, and the other GPT pairs with H_{10} on two documents per benchmark; Table 1 marks the cells computed on part of the test documents. Claude models ran through the Claude Agent SDK on a subscription and GPT through OpenRouter; providers' defaults and versions may change. The controls of §5.2 and the TimeML link step were run for one or two rounds only.

Cost. An annotation with H_{10} takes about three model calls per sentence (more with tool calls). With GPT-6.1-sol in both roles it cost about $0.07 per sentence at OpenRouter's prices, so the Chinese and English part of the shared task's test set (431 sentences, of which our test sets are 194) would cost about $31 with GPT; with Haiku on a subscription it costs no money but about $0.22–0.30 per sentence at API-equivalent prices. Sentences of a document are annotated in order (each sees the five before it), so the longest document (100 sentences) bounds the wall-clock time at about two to three hours; the ten rounds of one trajectory, with a test after each, took about three hours.

Data. The English training groups come from only three fully annotated documents; most of the released English UMR data are partial conversions without aspect or document-level relations. The Chinese gold we score against differs from the shared task's own gold (Appendix A), so we re-

port both. The TimeML results depend strongly on which gold the builder sees (§5.4). Test documents may occur in the models' pretraining data.

Admission. Operations are admitted by machine checks only; nothing tests an edit before it is kept, so a round can make the harness worse on unseen data (Figure 2). A held-in check of each edit, and people adjudicating the gold inconsistencies the builder reports, are natural additions.

Human influence. The seed builder's prompt, the reflection prompt, the operation set, the judges and the admission rules were designed by the authors. During the runs we changed the admission code twice (rule steps get one repair attempt; helper functions are allowed in tools) and restarted the trajectories from scratch each time, and we replaced the English and TimeML training pools once after finding training documents without document-level annotation.

Ethics Statement

Automating annotation can reduce the paid work available to annotators. We see ANNOFORGE as a way to start an annotation project from a working draft annotator, with people deciding conventions and adjudicating the inconsistencies the builder surfaces. UMR data are used under their CC licenses; TimeBank and AQUAINT text (LDC) is not redistributed. Model calls went to commercial APIs. Generative AI was used to run the experiments and to draft and edit this paper.

References

Steven Bethard. 2013. ClearTK-TimeML: A minimalist approach to TempEval 2013. In *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the Seventh International Workshop on Semantic Evaluation (SemEval 2013)*, pages 10–14, Atlanta, Georgia, USA. Association for Computational Linguistics.

Julia Bonn, Claire Bonial, Matt Buchholz, and 1 others. 2026. Uniform meaning representation 2.2. LINDAT/CLARIAH-CZ digital library at the Institute of Formal and Applied Linguistics (ÚFAL), Faculty of Mathematics and Physics, Charles University (publisher: UMR Consortium). Includes the DMR 2026 shared-task training data, gold test data and participating systems' outputs.

Julia Bonn, Matthew Buchholz, Jayeol Chun, Andrew Cowell, William Croft, Lukas Denk, Sijia Ge, Jan Hajič, Kenneth Lai, James H. Martin, Skatje Myers, Alexis Palmer, Martha Palmer, Claire Benét Post, James Pustejovsky, Kristine Stenzel, Haibo Sun, Zdeňka Urešová, Rosa Vallejos, and 4 others. 2024. Building a broad infrastructure for uniform meaning representations. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 2537–2547, Torino, Italia. ELRA and ICCL.

Jayeol Chun and Nianwen Xue. 2024. Uniform meaning representation parsing as a pipelined approach. In *Proceedings of TextGraphs-17: Graph-based Methods for Natural Language Processing*, pages 40–52.

Rémi de Vergnette and Maxime Amblard. 2026. Sema system for the DMR 2026 shared task: Multistage UMR parsing with Qwen3-4B. In *Proceedings of The Seventh International Workshop on Designing Meaning Representations (DMR 2026) @ LREC 2026*, pages 155–159, Palma, Mallorca (Spain). ELRA Language Resources Association (ELRA).

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Xingwei He, Zhenghao Lin, Yeyun Gong, A-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 165–190.

Johannes Heinecke and Munshi Asadullah. 2026. Orange @ UMR parsing shared task. In *Proceedings of The Seventh International Workshop on Designing Meaning Representations (DMR 2026) @ LREC 2026*, pages 148–154, Palma, Mallorca (Spain). ELRA Language Resources Association (ELRA).

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. 2024. When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*, 12:1417–1440.

Kon Woo Kim, Jin-Dong Kim, and Akiko Aizawa. 2026. Refining and reusing annotation guidelines for LLM annotation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 37951–37964, San Diego, California, United States. Association for Computational Linguistics.

Siun Kim and Hyung-Jin Yoon. 2026. DiZiNER: Disagreement-guided instruction refinement via simulating pilot annotation for zero-shot named entity recognition. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17498–17519, San Diego, California, United States. Association for Computational Linguistics.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. arXiv:2603.28052. *Preprint*, arXiv:2603.28052.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. 2026. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. arXiv:2604.25850. *Preprint*, arXiv:2604.25850.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Hector Llorens, Estela Saquete, and Borja Navarro. 2010. TIPSem (English and Spanish): Evaluating CRFs and semantic roles in TempEval-2. In *Proceedings of the 5th International Workshop on Semantic Evaluation*, pages 284–291, Uppsala, Sweden. Association for Computational Linguistics.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Martha Palmer, Daniel Gildea, and Paul Kingsbury. 2005. The proposition bank: An annotated corpus of semantic roles. *Computational Linguistics*, 31(1):71–106.

Liudas Panavas, Sebastian Minus, Bradley Monton, Derek Ray, Suhaas Garre, Sushant Mehta, and Edwin Chen. 2026. HANDBOOK.md: A benchmark for long-context agentic instruction following. arXiv:2607.25398. *Preprint*, arXiv:2607.25398.

James Pustejovsky, José Castaño, Robert Ingria, Roser Saurí, Robert Gaizauskas, Andrea Setzer, Graham Katz, and Dragomir Radev. 2003. TimeML: Robust specification of event and temporal expressions in text. In *AAAI Spring Symposium 2003 (AAAI Technical Report SS-03-07)*.

James Pustejovsky, Marc Verhagen, Roser Saurí, Jessica Littman, Robert Gaizauskas, Graham Katz, Inderjeet Mani, Robert Knippen, and Andrea Setzer. 2006. TimeBank 1.2. Linguistic Data Consortium, Philadelphia. LDC Catalog No. LDC2006T08.

Roser Saurí, Jessica Littman, Bob Knippen, Robert Gaizauskas, Andrea Setzer, and James Pustejovsky. 2006. TimeML annotation guidelines, version 1.2.1. https://timeml.github.io/site/publications/timeMLdocs/annguide_1.2.1.pdf. Dated January 31, 2006.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 8634–8652.

Jan Štěpánek, Daniel Zeman, Markéta Lopatková, Federica Gamba, Hana Hledíková, and Nianwen Xue. 2026. First shared task on UMR parsing. In *Proceedings of The Seventh International Workshop on Designing Meaning Representations (DMR 2026) @ LREC 2026*, pages 136–147, Palma, Mallorca (Spain). ELRA Language Resources Association (ELRA).

Haibo Sun, Jayeol Chun, and Nianwen Xue. 2025. AnCast++: Document-level evaluation of graph-based meaning representations. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19642–19654.

Haibo Sun and Nianwen Xue. 2024. Anchor and broadcast: An efficient concept alignment approach for evaluation of semantic graphs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 1052–1062, Torino, Italia. ELRA and ICCL.

Naushad UzZaman and James Allen. 2011. Temporal evaluation. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 351–356, Portland, Oregon, USA. Association for Computational Linguistics.

Naushad UzZaman, Hector Llorens, Leon Derczynski, James Allen, Marc Verhagen, and James Pustejovsky. 2013. SemEval-2013 task 1: TempEval-3: Evaluating time expressions, events, and temporal relations. In *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the Seventh International Workshop on Semantic Evaluation (SemEval 2013)*, pages 1–9, Atlanta, Georgia, USA. Association for Computational Linguistics.

Jens E. L. Van Gysel, Meagan Vigus, Jayeol Chun, Kenneth Lai, Sarah Moeller, Jiarui Yao, Tim O'Gorman, Andrew Cowell, William Croft, Chu-Ren Huang, Jan Hajič, James H. Martin, Stephan Oepen, Martha Palmer, James Pustejovsky, Rosa Vallejos, and Nianwen Xue. 2021. Designing a uniform meaning representation for natural language processing. *KI - Künstliche Intelligenz*, 35(3–4):343–360.

Marc Verhagen, Roser Saurí, Tommaso Caselli, and James Pustejovsky. 2010. SemEval-2010 task 13:

TempEval-2. In *Proceedings of the 5th International Workshop on Semantic Evaluation*, pages 57–62, Uppsala, Sweden. Association for Computational Linguistics.

Nianwen Xue and Martha Palmer. 2009. Adding semantic roles to the Chinese Treebank. *Natural Language Engineering*, 15(1):143–172.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling (COLM)*.

Daniel Zeman and Federica Gamba. 2026. Word alignment-based evaluation of uniform meaning representations. arXiv:2603.26401. *Preprint*, arXiv:2603.26401.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. 2026. Self-harness: Harnesses that improve themselves. arXiv:2606.09498. *Preprint*, arXiv:2606.09498.

A Data

Each benchmark uses ten training groups of five consecutive sentences, in a fixed random order, one group per round. Training and test texts do not overlap, and the builder never sees test documents.

UMR test documents. The shared task's test set (Štěpánek et al., 2026), released with UMR 2.2 (Bonn et al., 2026), has 15 Chinese and 5 English documents. We use those whose gold we have in a cleaned version [state the source and cleanup of this gold]: Chinese zh-0021–0024 and zh-0027–0035 (not zh-0026, zh-0036) and English en-0005–0008 (not en-0000). Shuffled with seed 20261012 and taken whole while the total stays at most 100 sentences, the Chinese test documents are 0023, 0027, 0028, 0030, 0031, 0032 and 0035 (99 sentences) and the English ones are en-0005, 0006, 0007 and 0008 (95 sentences). Tokens are identical to the shared task's blind test files. Our cleaned gold and the shared task's own test gold (UMR 2.2) agree at juːmæʧ 99.0 on the English documents but only 71.7 on the Chinese ones, which differ in alignment and in a number of annotation decisions; the main metric (AnCast++ F) uses the cleaned gold, juːmæʧ the official gold. The two annotations of the same English text in the test set (en-0005 and en-0007) agree at juːmæʧ 81.8 with each other, a rough ceiling for any system on that text.

UMR training groups. Chinese: ten documents of 0001–0018, one random window of five sentences each. English: the release is mostly partial AMR-to-UMR conversions (few aspect attributes, no document-level relations) or Minecraft dialogues; only three documents not sharing text with the test set have full sentence and document-level annotation (english_umr-0578 = wsj_0003, 0579 = wsj_0004, 0580 = the Pear Story), and they give the ten non-overlapping windows. The shared task's clean English training files 0001 and 0003 are the texts of test documents en-0006 and en-0008, annotated again, so we exclude them.

TimeML. The same whole-document sampling procedure selects 7 of the 20 TempEval-3 platinum documents (UzZaman et al., 2013): CNN_20130322_1003, CNN_20130322_248, CNN_20130322_314, WSJ_20130321_1145, WSJ_20130322_804, bbc_20130322_1353 and nyt_20130321_cyprus (96 sentences, scored against the official keys). Sentences are split at sentence-final punctuation followed by a capital, a digit or a quote, and at blank lines, never inside a gold tag. Training groups come either from TBAQ-cleaned (TimeBank 1.2 (Pustejovsky et al., 2006) and AQUAINT, the TempEval-3 training data) or from the 13 platinum documents that are not test documents. A window is scored on its own text: its tags, and TLINKs between them or to the document creation time.

B The Judges

umrjudge. AnCast++ aligns each predicted node to at most one gold node. The judge then scores 25 checks in 11 groups over the aligned graphs; each check yields units with a verdict (correct, partial, wrong, missing, extra) and a credit, and reports F over its units.

Group	Checks
a nodes	`event_found`, `event_concept`, `entity_found`, `entity_concept`, `granularity`
b senses	`event_sense`
c attributes	`aspect`, `modstr`, `polarity`, `refer`, `other_attributes`
d arguments	`core_args_event`, `core_args_abstract`, `peripheral_event`, `peripheral_entity`
e role labels	`core_labels`, `peripheral_labels`
f names	`name_tokens`, `wiki`
g reentrancy	`reentrancy`
h–k document	`temporal`, `modal`, `coref_chain`, `coref_subset`, `doc_format`

Table 7: The checks of umrjudge.

timemljudge. Predicted and gold `TIMEX3` and `EVENT` tags are matched one-to-one by character overlap, largest first. Times: `timex_found`

(relaxed extent), `timex_extent` (exact extent of a matched pair), `timex_type`, `timex_value`. Events: `event_found`, `event_extent`, `event_class`, and the instance attributes `event_tense`, `event_aspect`, `event_polarity`, `event_modality`. Temporal links, with predicted ends mapped to their matched gold entities and B–A links turned around: for event–time in the same sentence and in different sentences, event–document time, event–event in the same and in different sentences, and time–time, a `pair` check (are the right pairs linked) and a `label` check (the relation of a pair linked on both sides). The first group follows TempEval-3's entity measures (UzZaman et al., 2013), the link categories extend those of TempEval-2's tasks C–F (Verhagen et al., 2010) with event–time links across sentences and time–time links; unlike the official temporal-awareness score (UzZaman and Allen, 2011), the link checks use no temporal closure.

C Prompts

Output format (UMR). Every annotating step receives this system message (the TimeML one is analogous): *"You annotate {Chinese|English} text in Uniform Meaning Representation (UMR), one sentence at a time. Output format. The annotation of sentence number N is written exactly as a .umr file writes one sentence, in three sections:* `# sentence level graph:` *(a PENMAN graph; every variable is sN followed by a letter and optional digits)* `# alignment:` *(one line per variable:* `variable: start-end`*, 1-based word positions;* `0-0` *when aligned to no word)* `# document level annotation: (sNs0 / sentence :temporal (. . .) :modal (. . .) :coref (. . .))`. *In the document level annotation, a and b are variables of this sentence or of earlier sentences, or constants; leave out a group that has no relation."*

Seed. The builder receives the description of the three steps, the output format and the whole guideline, and is asked for each step's instructions, "at most 8,000 characters, in English, organized as a compact checklist a model can follow: what to annotate and in which order, the decisions that matter most for this scheme and how the guidelines settle them, the closed lists of labels and values that apply, and the frequent mistakes to avoid".

	UMR zh (F)		UMR en (F)		TimeML (TLINK)	
	H_0	H_{10}	H_0	H_{10}	H_0	H_{10}
Guideline in the prompt, Haiku	26.5		58.3		29.8	
Guideline through tools, Haiku	19.9		50.9		27.1	
Guideline through tools, GPT	53.3		62.1		36.0	
Harness, Haiku / Haiku	43.1	51.4	40.5	52.3	37.8	36.9
Harness, Haiku / GPT	38.4	52.8	43.1	53.7	31.3	29.0
Harness, GPT / Haiku	42.4	53.2	42.4	55.3	34.3	38.1
Harness, GPT / GPT	45.0	52.4	42.7	57.1	32.8	n/a

Table 8: Two test documents per benchmark (22, 38 and 22 sentences), on which every generator / critic pair (Haiku, GPT-6.1-sol) was run with the seed and with H_{10} (built with Haiku), and the guideline baselines. TimeML is scored by the official scorer on the two documents.

Reflection. The reflection prompt (`af/rsi.py` in the code) describes the annotator, the resource files and the operations with their constraints, then gives the current harness, the history of earlier rounds (training scores, applied and rejected operations), and the group's sentences with gold, every step's output and the judge's incorrect units. It ends: "problems: the error patterns of this group, most costly first, each with the step it should be fixed in. reflection: why they happen, and what you will change and why. operations: the changes, in the order to apply them. suggestions: anything else." Suggestions are never implemented, and the prompt says so.

D Reproduction

All code, harness versions, per-round reflections, operations, diffs and scores are in the experiment folder `experiment/annoforge` of the code repository [anonymize for review]; `af/run.py` annotates and scores one harness version, `af/rsi.py` runs a trajectory, `af/sota.py` re-scores earlier systems, and `analysis/annoforge.py` in the paper repository makes every table and figure. Claude models ran through the Claude Agent SDK, GPT-6.1-sol through OpenRouter; every answer is cached, so a run can be replayed exactly. TimeBank and AQUAINT text is not redistributed.

E Additional Results

Table 8 compares every generator/critic pair on two test documents per benchmark, with the guideline baselines on the same documents; Table 9 gives the two-round RSI runs with restricted operations (§5.4).

RSI with		UMR Chinese F	Comp.	juːmæʧ	UMR English F	Comp.	juːmæʧ	tools/rules zh	en
all operations	H_1	47.2	42.4	42.0	44.4	28.4	34.3	0/1	1/0
	H_2	51.3	46.6	42.2	49.3	30.7	36.8	1/1	1/0
no tools	H_1	45.5	41.4	33.0	42.9	26.8	33.8	0/0	0/0
	H_2	48.6	44.0	36.9	49.7	32.6	35.6	0/0	0/0
no rule steps	H_1	50.8	45.8	36.3	43.3	27.4	33.2	1/0	1/0
	H_2	51.6	48.8	45.1	47.0	28.7	36.4	2/0	1/0

Table 9: RSI with restricted operations (two rounds, Haiku / Haiku, from the same seed and training groups). tools/rules: tools and rule steps admitted so far.

F Round-by-Round Records

Tables 10–12 list every round of the three main trajectories: the main metric of the training group before and after the round (F for UMR, TLINK F1 for TimeML), the test score of the new version, the operations applied out of those proposed, the total size of the skills, and the number of model steps, rule steps and tools.

	Group n H_{n-1}	H_n	Test H_n	Ops applied	Skills (K chars)	Model steps / rules / tools
H_0	–	–	46.4	–	17.4	3 / 0 / 0
H_1	56.7	73.7	47.2	4/4	13.0	3 / 1 / 0
H_2	42.1	71.5	51.3	4/4	12.0	3 / 1 / 1
H_3	64.4	82.2	52.6	10/10	15.6	3 / 2 / 1
H_4	58.6	76.0	52.2	5/5	17.8	3 / 3 / 1
H_5	58.2	70.2	53.5	5/6	18.9	3 / 4 / 1
H_6	59.6	69.5	52.3	18/18	19.6	3 / 5 / 1
H_7	66.5	71.1	52.5	9/11	20.1	3 / 6 / 1
H_8	68.7	83.8	53.2	9/9	21.7	3 / 7 / 1
H_9	53.7	67.2	53.2	12/13	23.6	3 / 8 / 1
H_{10}	44.3	56.3	52.4	8/8	26.4	3 / 10 / 1

Table 10: Rounds of ANNOFORGE on Chinese UMR (AnCast++ F).

	Group n H_{n-1}	H_n	Test H_n	Ops applied	Skills (K chars)	Model steps / rules / tools
H_0	–	–	40.9	–	17.9	3 / 0 / 0
H_1	27.6	91.1	44.4	4/6	14.4	3 / 0 / 1
H_2	28.2	51.0	49.3	14/15	17.0	3 / 0 / 1
H_3	43.5	51.3	51.0	11/11	18.4	3 / 1 / 1
H_4	42.5	63.1	50.6	4/5	19.3	3 / 1 / 1
H_5	38.9	54.1	52.3	6/6	21.8	3 / 2 / 1
H_6	48.4	68.8	53.2	10/11	24.8	3 / 3 / 1
H_7	76.3	88.6	53.5	7/7	26.3	3 / 3 / 1
H_8	52.4	60.4	51.7	7/9	26.0	3 / 3 / 1
H_9	58.9	68.0	51.7	7/8	26.9	3 / 3 / 1
H_{10}	61.9	73.7	52.2	14/14	27.9	3 / 4 / 1

Table 11: Rounds of ANNOFORGE on English UMR (AnCast++ F).

	Group n H_{n-1}	H_n	Test H_n	Ops applied	Skills (K chars)	Model steps / rules / tools
H_0	–	–	39.5	–	17.2	3 / 0 / 0
H_1	57.8	61.8	38.7	13/15	19.9	3 / 0 / 0
H_2	17.8	40.0	40.3	11/11	21.3	3 / 0 / 0
H_3	36.7	57.0	38.6	17/17	22.5	3 / 0 / 0
H_4	32.4	50.0	37.7	13/15	23.6	3 / 0 / 0
H_5	47.1	47.9	35.4	10/11	24.2	3 / 0 / 0
H_6	45.7	71.8	39.1	13/13	24.7	3 / 0 / 0
H_7	22.4	37.5	37.3	14/15	25.7	3 / 0 / 0
H_8	11.1	50.0	36.4	10/11	27.3	3 / 0 / 0
H_9	57.8	71.6	39.7	11/16	28.3	3 / 0 / 0
H_{10}	39.8	40.8	39.7	9/10	28.8	3 / 0 / 0

Table 12: Rounds of ANNOFORGE on TimeML, trained on platinum documents (TLINK F1).

Group	Check	Units	H_0	H_{10}	Δ
a	a_event_found	471	94.4	96.6	+2.2
a	a_event_concept	421	81.2	84.9	+3.6
a	a_entity_found	1278	89.5	93.7	+4.2
a	a_entity_concept	1036	65.9	81.2	+15.3
a	a_granularity	1308	83.6	96.0	+12.3
b	b_event_sense	322	87.6	93.8	+6.2
c	c_aspect	392	74.5	74.8	+0.4
c	c_polarity	26	76.2	76.2	+0.0
c	c_refer	11	0.0	33.3	+33.3
c	c_other_attributes	280	52.6	69.9	+17.2
d	d_core_args_event	333	51.8	63.5	+11.6
d	d_core_args_abstract	50	39.2	35.2	−4.0
d	d_peripheral_event	191	35.0	40.6	+5.6
d	d_peripheral_entity	354	43.3	48.5	+5.2
e	e_core_labels	374	78.3	87.1	+8.8
e	e_peripheral_labels	309	77.3	76.7	−0.7
f	f_name_tokens	191	58.1	56.2	−1.9
f	f_wiki	97	0.0	–	–
g	g_reentrancy	340	30.0	35.5	+5.5
h	h_temporal	431	3.0	14.7	+11.6
i	i_modal	507	0.0	49.8	+49.8
j	j_coref_chain	275	13.8	17.5	+3.6
j	j_coref_subset	4	0.0	0.0	+0.0
k	k_doc_format	1437	23.0	90.5	+67.5

Table 13: umrjudge checks on the Chinese test documents, H_0 and H_{10} (Haiku / Haiku).

G Per-Check Scores

H Case Study Excerpts

The builder's reflection, English round 1 (verbatim, abridged).

> "Most of the lost score is format, not understanding. The skills taught a vocabulary (`:ref-person`, `Singular`, `:modstr`, `:modpred`, `:Str`, `DCT`) that the gold and the judge do not use. The critique step then reinforced it, and it even moved the temporal relation the wrong way. I will change four things. (1) I rewrite all three skills to use the gold conventions [...] (2) I add a deterministic rule step [...] that normalises attribute names and cases, converts `:modstr` into the document-level modal labels and then drops it [...] (3) I add a `roleset_lookup` tool over `propbank-en-args.txt`, so haiku picks real senses and roles instead of guessing [...] (4) I rewrite the critic to check against the same conventions."

The rule steps of (2) were rejected: after one repair

Group	Check	Units	H_0	H_{10}	Δ
a	a_event_found	546	90.5	93.6	+3.1
a	a_event_concept	451	76.1	80.1	+4.1
a	a_entity_found	817	89.0	88.4	−0.6
a	a_entity_concept	655	77.4	80.2	+2.8
a	a_granularity	1046	88.4	93.9	+5.5
b	b_event_sense	315	66.7	80.1	+13.4
c	c_aspect	365	64.3	59.6	−4.7
c	c_polarity	31	45.0	61.9	+16.9
c	c_refer	114	0.0	50.0	+50.0
c	c_other_attributes	495	11.4	56.1	+44.7
d	d_core_args_event	341	41.6	54.5	+13.0
d	d_core_args_abstract	55	38.7	43.8	+5.1
d	d_peripheral_event	150	25.2	28.8	+3.7
d	d_peripheral_entity	174	42.7	43.4	+0.6
e	e_core_labels	366	76.8	84.0	+7.2
e	e_peripheral_labels	162	64.2	70.1	+5.9
f	f_name_tokens	143	71.7	79.2	+7.5
f	f_wiki	127	71.0	66.7	−4.3
g	g_reentrancy	354	18.9	33.0	+14.1
h	h_temporal	418	4.8	13.1	+8.3
i	i_modal	520	0.0	32.0	+32.0
j	j_coref_chain	427	17.2	17.4	+0.2
j	j_coref_subset	62	0.0	0.0	+0.0
k	k_doc_format	1239	24.7	86.2	+61.5

Table 14: umrjudge checks on the English test documents.

Group	Check	Units	H_0	H_{10}	Δ
a timex	timex_found	62	89.3	90.1	+0.8
a timex	timex_extent	50	92.0	90.0	−2.0
a timex	timex_type	50	94.0	98.0	+4.0
a timex	timex_value	50	88.0	88.0	+0.0
b event	event_found	384	83.8	84.3	+0.5
b event	event_extent	277	100.0	99.6	−0.4
b event	event_class	277	83.0	85.2	+2.2
b event	event_tense	277	68.2	64.8	−3.5
b event	event_aspect	277	88.8	90.2	+1.3
b event	event_polarity	277	100.0	99.6	−0.4
b event	event_modality	277	95.3	96.2	+0.9
c tlink E-T	tlink_et_same_pair	66	74.3	79.6	+5.3
c tlink E-T	tlink_et_same_label	39	89.7	87.2	−2.6
c tlink E-T	tlink_et_other_pair	15	0.0	0.0	+0.0
d tlink E-DCT	tlink_edct_pair	188	34.4	44.8	+10.4
d tlink E-DCT	tlink_edct_label	39	84.6	81.2	−3.4
e tlink E-E	tlink_ee_same_pair	230	43.0	54.5	+11.5
e tlink E-E	tlink_ee_same_label	63	74.6	65.5	−9.1
e tlink E-E	tlink_ee_other_pair	122	6.3	11.9	+5.6
e tlink E-E	tlink_ee_other_label	4	75.0	50.0	−25.0
f tlink T-T	tlink_tt_pair	15	0.0	0.0	+0.0

Table 15: timemljudge checks on the TimeML test documents (platinum-trained trajectory).

attempt they still came with fewer than three tests. The builder added rule steps that passed in later rounds.

A tool and its revision (English). Round 1, the whole tool:

```
def roleset_lookup(lemma: str) -> str:
  """Look up English PropBank rolesets for a lemma
[. . . ]. Returns lines like 'dump-01 ARG0: ...
ARG1: ...' with all senses and their numbered
roles [. . . ]."""
  lem = lemma.strip().lower().replace(' ',
'-').replace('_', '-')
  text =
resources.read_resource('propbank-en-args.txt')
  out = [ln for ln in text.splitlines() if
ln.startswith(lem + '-')]
  return '\n'.join(out[:14]) if out else 'no
roleset for ' + lem
```

Round 9, reason given: "Show sense names (meaning) and roles from the XML, with fallback to base verb for derived nouns/adjectives, so the model can choose the right sense (consider-01 vs -02)." The new version reads `propbank-en/<lemma>.xml`, prints each roleset as `consider-02 [name] ARG0: . . .` , and tries *-ers*, *-ion*, *-ing* and other suffix variants when the lemma has no file.

A rule step (Chinese, round 10, the whole function).

```
def run(annotation: str, sentence: dict) -> str:
  """Ensure the document-level :modal group
contains (root :modal author), creating the group
if absent."""
  [. . . ] if PAT.search(doc): return annotation
  m = re.search(r':modal\s*\(', doc)
  if m: doc = doc[:m.end()] + '(root :modal
author)\n' + doc[m.end():]
  else: [add a :modal group with it] . . .
```

A Chinese tool answering with a convention. `lookup_frame(lemma)` returns the rolesets of a Chinese PropBank frame file, and when there is none: "NO FRAME for *lemma*: use *lemma*-00 with `:actor` (agent) / `:undergoer` (patient) instead of ARGn", the UMR guideline's convention for predicates without a frame file, attached by the builder to the tool's answer.